



第1章 Hibernate_day02

今日任务

- 案例一：使用 Hibernate 完成 CRM 客户管理中查询所由客户的操作

教学导航

教学目标	
教学方法	案例驱动法

案例一：使用 **Hibernate** 完成 **CRM** 客户管理中 查询所由客户的操作

1.1 案例需求：

1.1.1 需求描述

CRM 系统中客户信息管理模块功能包括：

- 新增客户信息
- 客户信息查询
- 修改客户信息
- 删除客户信息

1.2 相关知识点：

1.2.1 Hibernate 持久化类的编写规则

1.2.1.1 什么是持久化类：

Hibernate 是持久层的 ORM 映射框架，专注于数据的持久化工作。所谓的持久化，就是将内存



中的数据永久存储到关系型数据库中。那么知道了什么是持久化，什么又是持久化类呢？其实所谓的持久化类指的是一个 Java 类与数据库表建立了映射关系，那么这个类称为是持久化类。其实你可以简单的理解为持久化类就是一个 Java 类有了一个映射文件与数据库的表建立了关系。那么我们在编写持久化类的时候有哪些要求呢？接下来我们来看一下：

1.2.1.2 持久化类的编写规则：

我们在编写持久化类的时候需要有以下几点需要注意：

- **持久化类需要提供无参数的构造方法。**因为在 Hibernate 的底层需要使用反射生成类的实例。
- **持久化类的属性需要私有，对私有的属性提供公有的 get 和 set 方法。**因为在 Hibernate 底层会将查询到的数据进行封装。
- **持久化类的属性要尽量使用包装类的类型。**因为包装类和基本数据类型的默认值不同，包装类的类型语义描述更清晰而基本数据类型不容易描述。举个例子：

假设表中有一列员工工资，如果使用 double 类型，如果这个员工工资忘记录入到系统中，系统会将默认值 0 存入到数据库，如果这个员工工资被扣完了，也会向系统中存入 0。那么这个 0 就有了多重含义，而如果使用包装类类型就会避免以上情况，如果使用 Double 类型，忘记录入工资就会存入 null，而这个员工工资被扣完了，就会存入 0，不会产生歧义。

- **持久化类要有一个唯一标识 OID 与表的主键对应。**因为 Hibernate 中需要通过这个唯一标识 OID 区分在内存中是否是同一个持久化类。在 Java 中通过地址区分是否是同一个对象的，在关系型数据库的表中是通过主键区分是否同一条记录。那么 Hibernate 就是通过这个 OID 来进行区分的。Hibernate 是不允许在内存中出现两个 OID 相同的持久化对象的。
- **持久化类尽量不要使用 final 进行修饰。**因为 Hibernate 中有延迟加载的机制，这个机制中会产生代理对象，Hibernate 产生代理对象使用的是字节码的增强技术完成的，其实就是产生了当前类的一个子类对象实现的。如果使用了 final 修饰持久化类。那么就不能产生子类，从而就不会产生代理对象，那么 Hibernate 的延迟加载策略（是一种优化手段）就会失效。

持久化类我们已经可以正常编写了，但是在持久化类中需要有一个唯一标识 OID 与表的主键去建立映射关系。而且主键一般我们是不会让客户手动录入的，一般我们是由程序生成主键。那么 Hibernate 中也提供了相应的主键生成的方式，那么我们来看下 Hibernate 的主键生成策略。

1.2.2 Hibernate 主键生成策略

1.2.2.1 主键的类型：

在讲解 Hibernate 的主键生成策略之前，先来了解两个概念，即自然主键和代理主键，具体如下：

- **自然主键：**把具有业务含义的字段作为主键，称之为自然主键。例如在 customer 表中，如果把 name 字段作为主键，其前提条件必须是：每一个客户的姓名不允许为 null，不允许客户重名，并且不允许修改客户姓名。尽管这也是可行的，但是不能满足不断变化的业务需求，一旦出现了允许客户重名的业务需求，就必须修改数据模型，重新定义表的主键，这给数据库的维护增加了难度。

- **代理主键**：把不具备业务含义的字段作为主键，称之为代理主键。该字段一般取名为“ID”，通常为整数类型，因为整数类型比字符串类型要节省更多的数据库空间。在上面例子中，显然更合理的方式是使用代理主键。

1.2.2.2 Hibernate 的主键生成策略：

Hibernate 中，提供了几个内置的主键生成策略，其常用主键生成策略的名称和描述如下

主键生成策略	
名称	描述
increment	用于 long、short、或 int 类型，由 Hibernate 自动以递增的方式生成唯一标识符，每次增量为 1。只有当没有其它进程向同一张表中插入数据时才可以使用，不能在集群环境下使用。适用于代理主键。
identity	采用底层数据库本身提供的主键生成标识符，条件是数据库支持自动增长数据类型。在 DB2、MySQL、MS SQL Server、Sybase 和 HypersonicSQL 数据库中可以使用该生成器，该生成器要求在数据库中将主键定义成为自增长类型。适用于代理主键。
sequence	Hibernate 根据底层数据库序列生成标识符。条件是数据库支持序列。适用于代理主键。
native	根据底层数据库对自动生成表示符的能力来选择 identity、sequence、hilo 三种生成器中的一种，适合跨数据库平台开发。适用于代理主键。
uuid	Hibernate 采用 128 位的 UUID 算法来生成标识符。该算法能够在网络环境中生成唯一的字符串标识符，其 UUID 被编码为一个长度为 32 位的十六进制字符串。这种策略并不流行，因为字符串类型的主键比整数类型的主键占用更多的数据库空间。适用于代理主键。
assigned	由 java 程序负责生成标识符，如果不指定 id 元素的 generator 属性，则默认使用该主键生成策略。适用于自然主键。

1.2.3 Hibernate 的持久化对象的三种状态

1.2.3.1 持久化对象三种状态的概述

了解了主键的生成策略之后，我们可以进一步来了解持久化类了。Hibernate 为了更好的来管理持久化类，特将持久化类分成了三种状态。在 Hibernate 中持久化的对象可以划分为三种状态，分别是瞬时态、持久态和脱管态，一个持久化类的实例可能处于三种不同状态中的某一种，三种状态的详细介绍如下。

1、瞬时态（transient）



瞬时态也称为临时态或者自由态，瞬时态的实例是由 new 命令创建、开辟内存空间的对象，**不存在持久化标识 OID（相当于主键值），尚未与 Hibernate Session 关联**，在数据库中也**没有记录**，失去引用后将被 JVM 回收。瞬时状态的对象在内存中是孤立存在的，与数据库中的数据无任何关联，仅是一个信息携带的载体。

2、持久态（persistent）

持久态的对象**存在持久化标识 OID**，**加入到了 Session 缓存中**，并且相关联的 Session **没有关闭**，**在数据库中有对应的记录**，每条记录只对应唯一的持久化对象，需要注意的是，持久态对象是在事务还未提交前变成持久态的。

3、脱管态（detached）

脱管态也称离线态或者游离态，当某个持久化状态的实例与 Session 的关联被关闭时就变成了脱管态。**脱管态对象存在持久化标识 OID，并且仍然与数据库中的数据存在关联，只是失去了与当前 Session 的关联**，脱管状态对象发生改变时 Hibernate 不能检测到。

1.2.3.2 区分对象的三种状态:

为了帮助大家更好的理解持久化对象的三种状态，接下来通过具体的案例来演示持久化对象的三种状态。

```
@Test
// 测试三种状态
public void demo1(){
    Session session = HibernateUtils.openSession();
    Transaction tx = session.beginTransaction();

    Customer customer = new Customer();// 瞬时态对象: 没有持久化标识 OID, 没有被 session
    管理.
    customer.setCust_name("老王");

    Serializable id = session.save(customer);// 持久态对象: 有持久化标识 OID, 被 session
    管理.
    tx.commit();
    session.close();

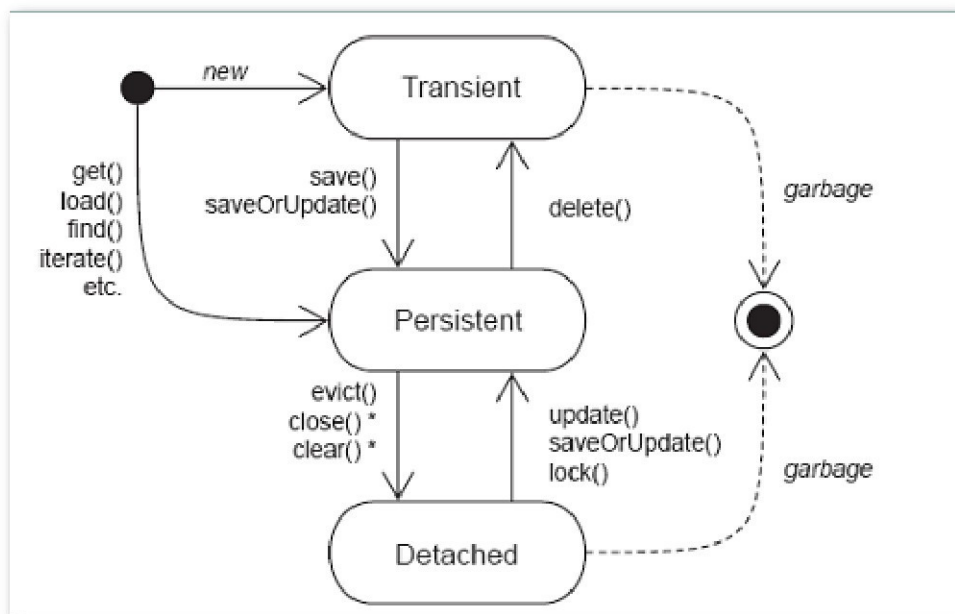
    System.out.println(customer);// 托管态对象: 有持久化标识 OID, 没有被 session 管理
}
```

customer 对象由 new 关键字创建，此时还未与 Session 进行关联，它的状态称为瞬时态；在执行了 session.save(customer)操作后，book 对象纳入了 Session 的管理范围，这时的 customer 对象变成了持久态对象，此时 Session 的事务还没有被提交；程序执行完 commit()操作并关闭了 Session 后，customer 对象与 Session 的关联被关闭，此时 customer 对象就变成了脱管态。

至此，便介绍完持久化对象的三种状态。在 Hibernate 的学习过程中，三种状态的特征变化以及各自的特点是非常重要的，有助于更好的理解 Hibernate，所以希望大家能够掌握。

1.2.3.3 Hibernate 持久化对象的三种状态转换

已经了解了 Hibernate 持久化对象的三种状态，然而这三种状态是可以通过一系列方法进行转换的，首先来看一下持久化对象的状态演化图



从图中可以看出，当一个对象被执行 new 关键字创建后，该对象处于瞬时态；当对瞬时态对象执行 Session 的 save()或 saveOrUpdate()方法后，该对象将被放入 Session 的一级缓存（在后面会有介绍），对象进入持久态；当对持久态对象执行 evict()、close()或 clear()操作后，对象进入脱管态；当直接执行 Session 的 get()、load()、find()或 iterate()等方法从数据库里查询对象时，查询到的对象也处于持久态；当对数据库中的纪录进行 update()、saveOrUpdate()以及 lock()等操作后，此时脱管态的对象就过渡到持久态；由于瞬时态和脱管态的对象不在 session 的管理范围，所以会在一段时间后被 JVM 回收。

持久化对象的三种状态可以通过调用 Session 中的一系列方法实现状态间的转换，具体如下：

1、瞬时态转换到其他状态

通过前面学习可知，瞬时态的对象由 new 关键字创建，瞬时态对象转换到其他状态总结如下：

- 瞬时态转换为持久态：执行 Session 的 save()或 saveOrUpdate()方法。
- 瞬时态转换为脱管态：为瞬时态对象设置持久化标识 OID。

由于持久化对象状态演化图中没有涉及到瞬时态转换到脱管态的情况，这里做下简要的说明，在前面学习中可知，脱管态对象存在 OID，但是没有 Session 的关联，也就是说脱管态和瞬时态的区别就是 OID 有没有值，所以可以通过为瞬时态对象设置 OID，使其变成脱管态对象。

```
Customer customer = new Customer(); // 瞬时态
customer.setCust_id(1); // 脱管态
```

2、持久态对象转换到其他状态

持久化对象可以直接通过 Hibernate 中 Session 的 get()、load()方法，或者 Query 查询从数据库中获取，持久态对象转换到其他状态总结如下：

- 持久态转换为瞬时态：执行 Session 的 delete()方法，需要注意的是被删除的持久化对象，不

建议再次使用。

- 持久态转换为脱管态：执行 Session 的 `evict()`、`close()`或 `clear()`方法。`evict()`方法用于清除一级缓存中某一个对象；`close()`方法用于关闭 Session，清除一级缓存；`clear()`方法用于清除一级缓存的所有对象。

3、脱管态对象转换到其他状态

脱管态对象无法直接获得，是由其他状态对象转换而来的，脱管态对象转换到其他状态总结如下：

- 脱管态转换为持久态：执行 Session 的 `update()`、`saveOrUpdate()`或 `lock()`方法。
- 脱管态转换为瞬时态：将脱管态对象的持久化标识 `OID` 设置为 `null`。

由于持久化对象状态演化图中没有涉及到脱管态转换到瞬时态的情况，这里做下简要的说明，跟瞬时态转换到脱管态的情况相似，脱管态和瞬时态的区别就是 `OID` 有没有值，所以可以通过将脱管态对象的 `OID` 设置为 `null`，使其变成瞬时态对象。例如在 `session.close()`操作后，加入代码 `customer.setCust_id(null);`，`customer` 对象将由脱管态转换为瞬时态。

1.2.3.4 持久态对象能够自动更新数据库

我们之前已经介绍了持久化对象的三种状态了，其实我们主要去研究持久态对象就够了，持久态对象其实有一个非常重要的特性：持久态对象可以自动更新数据库。

```
@Test
// 测试持久化类的持久态对象有自动更新数据库的能力
public void demo2(){
    Session session = HibernateUtils.openSession();
    Transaction tx = session.beginTransaction();

    // 获得持久态的对象.
    Customer customer = session.get(Customer.class, 11); // 持久态对象.
    customer.setCust_name("王五");

    // session.update(customer); // 不用手动调用 update 方法就可以更新

    tx.commit();
    session.close();
}
```

执行测试我们会发现，我们并没有手动调用 `update` 方法，Hibernate 就可以将数据自动更新了。持久态对象就有这样的一个功能。持久态对象之所以有这样的功能其实都依赖了 Hibernate 的一级缓存。接下来我们就开始学习 Hibernate 的一级缓存。

1.2.4 Hibernate 的一级缓存

缓存是计算机领域非常通用的概念。它介于应用程序和永久性数据存储源(如硬盘上的文件或者数据库)之间，其作用是降低应用程序直接读写永久性数据存储源的频率，从而**提高应用的运行性能**。缓存中的数据是数据存储源中数据的拷贝。缓存的物理介质通常是内存。



Hibernate 的缓存分为一级缓存和二级缓存，Hibernate 的这两级缓存都位于持久化层，存储的都是数据库数据的备份。其中第一级缓存为 Hibernate 的内置缓存，不能被卸载。接下来围绕 Hibernate 的一级缓存进行详细地讲解。

1.2.4.1 什么是 Hibernate 的一级缓存

Hibernate 的一级缓存就是指 Session 缓存，Session 缓存是一块内存空间，用来存放相互管理的 java 对象，在使用 Hibernate 查询对象的时候，首先会使用对象属性的 OID 值在 Hibernate 的一级缓存中进行查找，如果找到匹配 OID 值的对象，就直接将该对象从一级缓存中取出使用，不会再查询数据库；如果没有找到相同 OID 值的对象，则会去数据库中查找相应数据。当从数据库中查询到所需数据时，该数据信息也会放置到一级缓存中。**Hibernate 的一级缓存的作用就是减少对数据库的访问次数。**

在 Session 接口的实现中包含一系列的 **Java 集合**，这些 Java 集合构成了 Session 缓存。只要 Session 实例没有结束生命周期，存放在它缓存中的对象也不会结束生命周期。固一级缓存也被称为是 Session 基本的缓存。

Hibernate 的一级缓存有如下特点：

- 当应用程序调用 Session 接口的 save()、update()、saveOrUpdate 时，如果 Session 缓存中没有相应的对象，Hibernate 就会自动的把从数据库中查询到的相应对象信息加入到一级缓存中去。
- 当调用 Session 接口的 load()、get()方法，以及 Query 接口的 list()、iterator()方法时，会判断缓存中是否存在该对象，有则返回，不会查询数据库，如果缓存中没有要查询对象，再去数据库中查询对应对象，并添加到一级缓存中。
- 当调用 Session 的 close()方法时，Session 缓存会被清空。

1.2.4.2 测试一级缓存：

我们已经大致了解了什么是一级缓存了，那么一级缓存具体是否存在呢，我们可以通过如下的程序来证明一级缓存是存在的。

```
@Test
// 证明 Hibernate 的一级缓存的存在：
public void demo1() {
    Session session = HibernateUtils.openSession();
    Transaction tx = session.beginTransaction();

    Customer customer1 = session.get(Customer.class, 11); // 马上发生一条 sql 查询 1 号
    客户.并将数据存入了一级缓存
    System.out.println(customer1);

    Customer customer2 = session.get(Customer.class, 11); // 没有发生 SQL 语句从一级缓
```




存中获取数据。

```
System.out.println(customer2);

System.out.println(customer1 == customer2); // true 一级缓存缓存的是对象的地址。
tx.commit();
session.close();
}
```

在以上代码中，第一次执行 Session 的 get() 方法获取 customer1 对象时，由于一级缓存中没有数据，所以 Hibernate 会向数据库发送一条 sql 语句，查询 id 等于 1 的对象；当再次调用了 Session 的 get() 方法获取 customer2 对象时，将不会再发送 sql 语句，这是因为 customer2 对象是从一级缓存中获取的。

接下来，验证一下代码的执行结果是否和描述的相一致。在 Customer customer1 = session.get(Customer.class, 1l); 这一行设置断点，用 debug 方式执行 demo1() 方法，程序进入断点后点击单步跳过（或者按 F6），代码执行过 System.out.println(customer1); 语句后，控制台的输出结果，如下图所示。

```
Console: HibernateDemo4.demo1 [JUnit] C:\Program Files (x86)\Java\jre7\bin\java.exe (2016年7月27日 上午11:46:21)
Hibernate:
select
  customer0_.cust_id as cust_id1_0_0_,
  customer0_.cust_name as cust_nam2_0_0_,
  customer0_.cust_source as cust_sou3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_lev5_0_0_,
  customer0_.cust_phone as cust_pho6_0_0_,
  customer0_.cust_mobile as cust_mob7_0_0_
from
  cst_customer customer0_
where
  customer0_.cust_id=?
Customer [cust_id=1, cust_name=传智播客, cust_source=网络推广, cust_industry=IT教育, cust_level=null, cust_phone=null, cust_mobile=null]
```

输出结果

从上图的输出结果中可以看出，执行第一次 session.get() 方法后 Hibernate 向数据库发送了一条 select 语句，这说明此时 customer1 对象是从数据库中查找的。执行输出语句输出 customer1 对象中的数据后，继续执行程序，当执行到输出 customer2 对象的代码处时，控制台输出结果如下图所示。

```
Console: HibernateDemo4.demo1 [JUnit] C:\Program Files (x86)\Java\jre7\bin\java.exe (2016年7月27日 上午11:46:21)
Hibernate:
select
  customer0_.cust_id as cust_id1_0_0_,
  customer0_.cust_name as cust_nam2_0_0_,
  customer0_.cust_source as cust_sou3_0_0_,
  customer0_.cust_industry as cust_ind4_0_0_,
  customer0_.cust_level as cust_lev5_0_0_,
  customer0_.cust_phone as cust_pho6_0_0_,
  customer0_.cust_mobile as cust_mob7_0_0_
from
  cst_customer customer0_
where
  customer0_.cust_id=?
Customer [cust_id=1, cust_name=传智播客, cust_source=网络推广, cust_industry=IT教育, cust_level=null, cust_phone=null, cust_mobile=null]
Customer [cust_id=1, cust_name=传智播客, cust_source=网络推广, cust_industry=IT教育, cust_level=null, cust_phone=null, cust_mobile=null]
```

输出结果

从上图的输出结果可以看出，customer2 对象的查询结果被直接打印了，说明第二次调用 Session 对象的 get() 方法没有向数据库发送 select 语句，而是直接从一级缓存中获取 customer2 对象。

之前我们介绍过 Hibernate 的持久态对象能够自动更新数据库，其实就是依赖了一级缓存，那么一级缓存为什么就可以去更新数据库了呢，其实是因为一级缓存的一块特殊的区域就是快照区。

1.2.4.3 一级缓存的内部结构: (快照区)

Hibernate 向一级缓存放入数据时,同时复制一份数据放入到 Hibernate 快照中,当使用 commit() 方法提交事务时,同时会清理 Session 的一级缓存,这时会使用 OID 判断一级缓存中的对象和快照中的对象是否一致,如果两个对象中的属性发生变化,则执行 update 语句,将缓存的内容同步到数据库,并更新快照;如果一致,则不执行 update 语句。Hibernate 快照的作用就是确保一级缓存中的数据和数据库中的数据一致。

一级缓存的快照区:

```
@Test
// 一级缓存的快照区:持久态的对象能够自动更新数据库。
public void demo2(){
    Session session = HibernateUtils.openSession();
    Transaction tx = session.beginTransaction();

    Customer customer = session.get(Customer.class, 11);
    customer.setCust_name("王冠希");

    tx.commit();
    session.close();
}
```

一级缓存区

session	SessionImpl (id=43)
actionQueue	ActionQueue (id=64)
persistenceContext	StatefulPersistenceContext (id=132)

缓存

map

快照

id:1
name:王如花
王冠希

id:1
name:王如花

比对缓存区和快照区的数据是否一致,如果一致不更新数据库,如果不一致自动更新数据库。

1.2.5 Hibernate 的事务控制

Hibernate 是对 JDBC 的轻量级封装,其主要功能是操作数据库。在操作数据库过程中,经常会遇到事务处理的问题,那么我们接下来就来介绍 Hibernate 中的事务管理。

在学习 Hibernate 中的事务处理之前,先来回顾一下什么是事务。

1.2.5.1 什么是事务

在数据库操作中,一项事务(Transaction)是由一条或多条操作数据库的 SQL 语句组成的一个不可分割的工作单元。当事务中的所有操作都正常完成时,整个事务才能被提交到数据库中,如果有一项操作没有完成,则整个事务会被回滚。

其实事务总结起来理解为:逻辑上的一组操作,组成这组操作的各个单元,要么一起成功,要么一起失败。

1.2.5.2 事务的四个特性

事务有很严格的定义,需要同时满足四个特性,即原子性、一致性、隔离性、持久性。这四个



特性通常称之为 ACID 特性，具体如下：

- 原子性（Atomic）：表示将事务中所做的操作捆绑成一个不可分割的单元，即对事务所进行的数据修改等操作，要么全部执行，要么全都不执行。
- 一致性（Consistency）：表示事务完成时，必须使所有的数据都保持一致状态。
- 隔离性（Isolation）：指一个事务的执行不能被其它事务干扰。即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。
- 持久性（Durability）：持久性也称永久性（permanence），指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。提交后的其他操作或故障不会对其有任何影响。

1.2.5.3 事务的并发问题

在实际应用过程中，数据库是要被多个用户所共同访问的。在多个事务同时使用相同的数据时，可能会发生并发的问题，具体如下。

（1）脏读：一个事务读取到另一个事务未提交的数据。

（2）不可重复读：一个事务读到了另一个事务已经提交的 update 的数据，导致在同一个事务中的多次查询结果不一致。

（3）虚读/幻读：一个事务读到了另一个事务已经提交的 insert 的数据，导致在同一个事务中的多次查询结果不一致。

1.2.5.4 事务的隔离级别

为了避免事务并发问题的发生，在标准 SQL 规范中，定义了 4 个事务隔离级别，不同的隔离级别对事务的处理不同。

- 读未提交（Read Uncommitted, 1 级）：一个事务在执行过程中，既可以访问其他事务未提交的新插入的数据，又可以访问未提交的修改数据。如果一个事务已经开始写数据，则另外一个事务则不允许同时进行写操作，但允许其他事务读此行数据。此隔离级别可防止丢失更新。
- 已提交读（Read Committed, 2 级）：一个事务在执行过程中，既可以访问其他事务成功提交的新插入的数据，又可以访问成功修改的数据。读取数据的事务允许其他事务继续访问该行数据，但是未提交的写事务将会禁止其他事务访问该行。此隔离级别可有效防止脏读。
- 可重复读（Repeatable Read, 4 级）：一个事务在执行过程中，可以访问其他事务成功提交的新插入的数据，但不可以访问成功修改的数据。读取数据的事务将会禁止写事务（但允许读事务），写事务则禁止任何其他事务。此隔离级别可有效的防止不可重复读和脏读。
- 序列化/串行化（Serializable, 8 级）：提供严格的事务隔离。它要求事务序列化执行，事务只能一个接着一个地执行，但不能并发执行。此隔离级别可有效的防止脏读、不可重复读和幻读。

隔离级别	含义
READ_UNCOMMITT	允许你读取还未提交的改变了的数据。可能导致脏、幻、不可重



ED	复读
READ_COMMITTED	允许在并发事务已经提交后读取。可防止脏读，但幻读和 不可重复读仍可发生
REPEATABLE_READ	对相同字段的多次读取是一致的，除非数据被事务本身改变。可防止脏、不可重复读，但幻读仍可能发生。
SERIALIZABLE	完全服从 ACID 的隔离级别，确保不发生脏、幻、不可重复读。这在所有的隔离级别中是最慢的，它是典型的通过完全锁定在事务中涉及的数据表来完成的。

事务的隔离级别，是由数据库提供的，并不是所有数据库都支持四种隔离级别

- MySQL：READ_UNCOMMITTED、READ_COMMITTED、REPEATABLE_READ、SERIALIZABLE（默认 REPEATABLE_READ）
- Oracle：READ_UNCOMMITTED、READ_COMMITTED、SERIALIZABLE（默认 READ_COMMITTED）

在使用数据库时候，隔离级别越高，安全性越高，性能越低。

实际开发中，不会选择最高或者最低隔离级别，选择 READ_COMMITTED（oracle 默认）、REPEATABLE_READ（mysql 默认）

1.2.5.5 Hibernate 中的事务管理

在 Hibernate 中，可以通过代码来操作管理事务，如通过“Transaction tx = session.beginTransaction();”开启一个事务；持久化操作后，通过“tx.commit();”提交事务；如果事务出现异常，又通过“tx.rollback();”操作来撤销事务（事务回滚）。

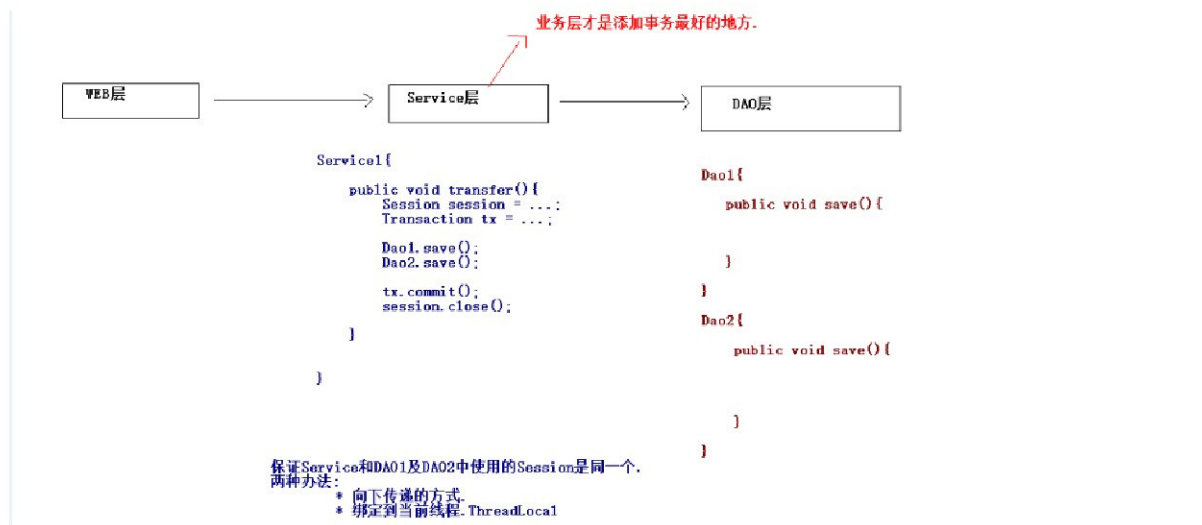
除了在代码中对事务开启，提交和回滚操作外，还可以在 Hibernate 的配置文件中对事务进行配置。配置文件中，可以设置事务的隔离级别。其具体的配置方法是在 hibernate.cfg.xml 文件中的 <session-factory> 标签元素中进行的。配置方法如下所示。

```
<!--
    事务隔离级别
    hibernate.connection.isolation = 4
    1—Read uncommitted isolation
    2—Read committed isolation
    4—Repeatable read isolation
    8—Serializable isolation
-->
<property name="hibernate.connection.isolation">4</property>
```

到这我们已经设置了事务的隔离级别，那么我们在真正进行事务管理的时候，需要考虑事务的



应用的场景，也就是说我们的事务控制不应该是在 DAO 层实现的，应该在 Service 层实现，并且在 Service 中调用多个 DAO 实现一个业务逻辑的操作。具体操作如下显示：



其实最主要的是如何保证在 Service 中开启的事务时使用的 Session 对象和 DAO 中多个操作使用的是同一个 Session 对象。

其实有两种办法可以实现：

- 可以在业务层获取到 Session，并将 Session 作为参数传递给 DAO。
- 可以使用 ThreadLocal 将业务层获取的 Session 绑定到当前线程中，然后在 DAO 中获取 Session 的时候，都从当前线程中获取。

其实使用第二种方式肯定是最优方案，那么具体的实现已经不用我们完成了，Hibernate 的内部已经将这个事情做完了。我们只需要完成一段配置即可。

Hibernate5 中自身提供了三种管理 Session 对象的方法

Session 对象的生命周期与本地线程绑定

Session 对象的生命周期与 JTA 事务绑定

Hibernate 委托程序管理 Session 对象的生命周期

在 Hibernate 的配置文件中，hibernate.current_session_context_class 属性用于指定 Session 管理方式，可选值包括

- thread: Session 对象的生命周期与本地线程绑定
- jta: Session 对象的生命周期与 JTA 事务绑定
- managed: Hibernate 委托程序来管理 Session 对象的生命周期

在 hibernate.cfg.xml 中进行如下配置：

```

<!-- 配置session绑定本地线程 -->
<property name="hibernate.current_session_context_class">thread</property>
    
```

hibernate 提供 sessionFactory.getCurrentSession() 创建一个 session 和 ThreadLocal 绑定方法。

在 HibernateUtil 工具类中更改 getCurrentSession 方法：

```

//获取当前线程绑定的会话
public static Session getCurrentSession(){
    
```



```
return sessionFactory.getCurrentSession();  
}
```

而且 Hibernate 中提供的这个与线程绑定的 session 可以不用关闭，当线程执行结束后，就会自动关闭了。

到这里我们已经对 Hibernate 的事务管理有了基本的了解，但是之前我们所做的 CRUD 的操作其实还没有查询多条记录。那如果我们需要查询多条记录要如何完成呢，我们接下来去学习一下 Hibernate 的其他的相关的 API。

1.2.6 Hibernate 的其他 API

1.2.6.1 Query

Query 代表面向对象的一个 Hibernate 查询操作。在 Hibernate 中，通常使用 session.createQuery() 方法接受一个 HQL 语句，然后调用 Query 的 list() 或 uniqueResult() 方法执行查询。所谓的 HQL 是 Hibernate Query Language 缩写，其语法很像 SQL 语法，但它是完全面向对象的。

在 Hibernate 中使用 Query 对象的步骤，具体所示：

- (1) 获得 Hibernate 的 Session 对象。
- (2) 编写 HQL 语句。
- (3) 调用 session.createQuery 创建查询对象。
- (4) 如果 HQL 语句包含参数，则调用 Query 的 setXxx 设置参数。
- (5) 调用 Query 对象的 list() 或 uniqueResult() 方法执行查询。

了解了使用 Query 对象的步骤后，接下来，通过具体示例来演示 Query 对象的查询操作。

Query 执行 HQL 语句：

```
// 1. 查询所有记录  
/*Query query = session.createQuery("from Customer");  
List<Customer> list = query.list();  
System.out.println(list);*/  
  
// 2. 条件查询：  
/*Query query = session.createQuery("from Customer where name = ?");  
query.setString(0, "张三");  
List<Customer> list = query.list();  
System.out.println(list);*/  
  
// 3. 条件查询：  
/*Query query = session.createQuery("from Customer where name = :aaa and age  
= :bbb");  
query.setString("aaa", "张三");  
query.setInteger("bbb", 38);  
List<Customer> list = query.list();  
System.out.println(list);*/
```




```
// 4. 分页查询:  
Query query = session.createQuery("from Customer");  
query.setFirstResult(3);  
query.setMaxResults(3);  
List<Customer> list = query.list();  
System.out.println(list);
```

程序通过使用 Query 接口，将 customer 表中的三条数据全部输出。更详细的 HQL 使用，会在后续讲解，此处只需了解 Hibernate 中是如何使用 Query 接口进行数据查询的即可。

Query 中除了使用 list() 方法查询全部数据外，还有其它一些常用方法，具体如下：

- setter 方法：Query 接口中提供了一系列的 setter 方法用于设置查询语句中的参数，针对不同的数据类型，需要用到不同的 setter 方法。
- iterator() 方法：该方法用于查询语句，返回的结果是一个 Iterator 对象，在读取时只能按照顺序方式读取，它仅把使用到的数据转换成 Java 实体对象。
- uniqueResult() 方法：该方法用于返回唯一的结果，在确保只有一条记录的查询时可以使用该方法。
- executeUpdate() 方法：该方法是 Hibernate3 的新特性，它支持 HQL 语句的更新和删除操作。
- setFirstResult() 方法：该方法可以设置获取第一个记录的位置，也就是它表示从第几条记录开始查询，默认从 0 开始计算。
- setMaxResult() 方法：该方法用于设置结果集的最大记录数，通常与 setFirstResult() 方法结合使用，用于限制结果集的范围，以实现分页功能。

1.2.6.2 Criteria

Criteria 是一个完全面向对象，可扩展的条件查询 API，通过它完全不需要考虑数据库底层如何实现，以及 SQL 语句如何编写，它是 Hibernate 框架的核心查询对象。Criteria 查询，又称为 QBC 查询（Query By Criteria），它是 Hibernate 的另一种对象检索方式。

org.hibernate.criterion.Criterion 是 Hibernate 提供的一个面向对象查询条件接口，一个单独的查询就是 Criterion 接口的一个实例，用于限制 Criteria 对象的查询，在 Hibernate 中 Criterion 对象的创建通常是通过 Restrictions 工厂类完成的，它提供了条件查询方法。

通常，使用 Criteria 对象查询数据的主要步骤，具体如下：

- (1) 获得 Hibernate 的 Session 对象。
- (2) 通过 Session 获得 Criteria 对象。
- (3) 使用 Restrictions 的静态方法创建 Criterion 条件对象。Restrictions 类中提供了一系列用于设定查询条件的静态方法，这些静态方法都返回 Criterion 实例，每个 Criterion 实例代表一个查询条件。
- (4) 向 Criteria 对象中添加 Criterion 查询条件。Criteria 的 add() 方法用于加入查询条件。
- (5) 执行 Criteria 的 list() 或 uniqueResult() 获得结果。

了解了 Criteria 对象的使用步骤后，接下来，通过具体示例来演示 Criteria 对象的查询操作。

Criteria: 用来执行条件查询

```
// 1. 查询所有记录
```



```
/*Criteria criteria = session.createCriteria(Customer.class);
List<Customer> list = criteria.list();
System.out.println(list);*/

// 2.条件查询
/*Criteria criteria = session.createCriteria(Customer.class);
criteria.add(Restrictions.eq("name", "李健"));
List<Customer> list = criteria.list();
System.out.println(list);*/

// 3.条件查询
/*Criteria criteria = session.createCriteria(Customer.class);
criteria.add(Restrictions.eq("name", "李健"));
criteria.add(Restrictions.eq("age", 38));
List<Customer> list = criteria.list();
System.out.println(list);*/

// 4.分页查询
Criteria criteria = session.createCriteria(Customer.class);
criteria.setFirstResult(3);
criteria.setMaxResults(3);
List<Customer> list = criteria.list();
System.out.println(list);
```

在 Criteria 对象中，除了使用 criteria.list()方法查询全部数据外，还有其它一些常用方法：如果只返回一个值时，可以使用 criteria 的 uniqueResult()方法；如果需要分页时可以使用 setFirstResult()和 setMaxResult()两个方法，setFirstResult()方法表示从第几条记录开始查询，setMaxResult()方法表示查询几条记录。

1.2.6.3 SQLQuery

SQLQuery 这个就比较简单了，这个接口用于接收一个 sql 语句进行查询，然后调用 list()或者 uniqueResult() 方法进行查询。但是 sql 语句不会直接封装到实体对象中，需要我们手动写代码才可以封装到实体中。

```
// 基本查询
/*SQLQuery sqlQuery = session.createSQLQuery("select * from cst_customer");
List<Object[]> list = sqlQuery.list();

for (Object[] objects : list) {
    System.out.println(Arrays.toString(objects));
}*/
```



```
SQLQuery sqlQuery = session.createSQLQuery("select * from cst_customer");  
// 封装到对象中  
sqlQuery.addEntity(Customer.class);  
List<Customer> list = sqlQuery.list();  
  
for(Customer customer:list){  
    System.out.println(customer);  
}
```